BÁO CÁO THỰC HÀNH

Thực hành An toàn mạng máy tính

Lab X: NỘI DUNG XYZ

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: dd/mm/yyyy

Nhóm 7 – NT101.R12.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24521664	Phạm Thanh Thiên	Làm chung	50%
24521708	Trương Đức Thịnh	Làm chung	50%

Link GitHub

Nhiệm vụ 2.1

Kiểm tra phiên bản Python và OpenSSL trong môi trường thực hành:

```
>> & $sslLab version
Python 3.14.4
OpenSSL 3.5.4 30 Sep 2025 (Library: OpenSSL 3.5.4 30 Sep 2025)
PS E:\AnToanMang\lab03\Lab03_RSA_X509>
```

Kết quả tính ba cặp khóa RSA từ các tham số đề bài:

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Python\pythoncore-3.14-64\python.exe' .\task21.py keys
2.1.1 - TẠO BA CẶP KHÓA

Khóa 1:
n = 187
phi = 160
e = 7
d = 23
PU = (e, n); PR = (d, n)
Số bit n = 8
(e*d) mod phi = 1

Khóa 2:
n = 135884761403422083943951664696476272266743485417 91
phi = 135884761403422083943951457129366086326159902197 84
e = 17
d = 799322125902482846729126218408035801918587659987 3
PU = (e, n); PR = (d, n)
Số bit n = 164
(e*d) mod phi = 1

Khóa 3:
n = 101776877529005912638346811918779931246783058062684819617574643018368103302097
phi = 101776877529005912638346811918779931246144673982897083685478381401683682492836
e = 886979
d = 24212225287904763939160097464943268930139828978795606022583874367720623008491
PU = (e, n); PR = (d, n)
Số bit n = 256
(e*d) mod phi = 1
```

Chương trình tính n=p×q và phi(n)=(p−1)(q−1), sau đó xác định d là nghịch đảo nhân của e theo modulo phi(n). Khóa công khai được biểu diễn bởi PU=(e,n), còn khóa riêng là PR=(d,n).

Với bộ tham số thứ nhất, kết quả thu được là n=187, phi(n)=160, e=7 và d=23. Do đó, khóa công khai là PU_1=(7,187) và khóa riêng là PR_1=(23,187). Hai bộ tham số còn lại tạo ra modulus có độ dài lần lượt là 164 bit và 256 bit. Trong cả ba trường hợp, chương trình đều kiểm tra được e×d mod phi(n)=1, xác nhận d được tính đúng theo điều kiện tạo khóa RSA.

Mã hóa và giải mã bản rõ M=5 theo hai hướng của RSA:

```
(e*d) mod phi = 1                    & 'C:\Users\Win\AppData\Local\Pytho
n\pythoncore-3.14-64\python.exe' .\task21.py number

2.1.2 - BẢN RÕ M = 5, KHÓA 1
Bảo mật: M = 5 -> C = 146 -> M = 5
Xác thực theo mô hình đề: M = 5 -> S = 180 -> M = 5
PS E:\AnToanMang\lab03\Lab03_RSA_X509>
```

Mã hóa thông điệp bằng ba khóa RSA, biểu diễn Base64 và kiểm tra kết quả giải mã:

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Pytho
n\pythoncore-3.14-64\python.exe' .\task21.py text

2.1.3 - MÃ HÓA CHUỖI, XUẤT BASE64
Bản rõ: The University of Information Technology

Khóa 1; khối rõ/mã (byte) = (1, 1); độ dài gốc = 40
Base64 = MrNUTERCYGVUfp1gSk1Mm3dMPUJ3m36DXEpgm0JMMlSws0KbMJuJTQ==
Giải mã = The University of Information Technology
Khôi phục đúng = True

Khóa 2; khối rõ/mã (byte) = (20, 21); độ dài gốc = 40
Base64 = AWcKXgyeSmY1g3ZElW7IsbcQSYADnjA2y6yY8Hu9900ymPVaONVGLa4A
Giải mã = The University of Information Technology
Khôi phục đúng = True

Khóa 3; khối rõ/mã (byte) = (31, 32); độ dài gốc = 40
Base64 = WaG19PwIkNUlVfdbuta2gEk0MtlxY+ZAhRmTc3sogdGEwROPCkKp9/Sb0MjUUGwmB
4F4s+zOU9zhzEy3+mMNXA==
Giải mã = The University of Information Technology
Khôi phục đúng = True
```

Thông điệp được sử dụng trong thực nghiệm là “The University of Information Technology”, có độ dài 40 byte khi mã hóa bằng UTF-8.

Chương trình chuyển thông điệp thành dãy byte và chia thành các khối sao cho giá trị số nguyên của mỗi khối nhỏ hơn modulus n.

Với ba khóa đã tạo, kích thước khối bản rõ lần lượt là 1, 20 và 31 byte. Mỗi khối được chuyển thành số nguyên theo thứ tự little-endian, sau đó mã hóa bằng công thức C=M^e mod n. Các khối bản mã được ghi với độ dài cố định tương ứng là 1, 21 và 32 byte, ghép lại rồi biểu diễn dưới dạng Base64.

Chương trình giải mã bằng khóa riêng theo công thức M=C^d mod n và chuyển dữ liệu trở lại thành văn bản. Kết quả của cả ba khóa đều khôi phục đúng thông điệp ban đầu. Base64 chỉ là cách biểu diễn dữ liệu bản mã bằng ký tự, không phải một thuật toán mã hóa bảo mật.

Kết quả giải mã bốn bản mã cho trước và kiểm tra bằng phép mã hóa lại:

```
2.1.4 - GIẢI BỐN BẢN MÃ CHO SẴN

Bản mã 1; Base64; 63 byte
Khóa 2; little-endian; khối mã = [21, 21, 21]
Bản rõ = Ron Rivest, Adi Shamir and Leonard Adleman
Mã hóa lại trùng bản mã = True

Bản mã 2; HEX; 20 byte
Khóa 2; little-endian; khối mã = [20]
Bản rõ = Meet-in-the-middle
Mã hóa lại trùng bản mã = True

Bản mã 3; Base64; 46 byte
Khóa 1; little-endian; khối mã = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1]
Bản rõ = Vietnam National University - Ho Chi Minh City
Mã hóa lại trùng bản mã = True

Bản mã 4; BIN; 21 byte
Khóa 2; little-endian; khối mã = [21]
Bản rõ = Man-in-the-middle
Mã hóa lại trùng bản mã = True
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Pytho
n\pythoncore-3.14-64\python.exe' .\task21.py decrypt
```

Chương trình chuyển các bản mã từ biểu diễn Base64, HEX hoặc nhị phân về dãy byte, sau đó thử giải mã bằng các khóa đã tính. Với quy ước little-

endian và cách chia khối phù hợp, bản mã thứ nhất, thứ hai và thứ tư được giải bằng khóa 2; bản mã thứ ba được giải bằng khóa 1.

Các bản rõ khôi phục lần lượt là “Ron Rivest, Adi Shamir and Leonard Adleman”, “Meet-in-the-middle”, “Vietnam National University - Ho Chi Minh City” và “Man-in-the-middle”. Chương trình mã hóa lại từng khối bản rõ và đối chiếu với bản mã đầu vào. Cả bốn trường hợp đều cho kết quả trùng khớp, củng cố tính đúng đắn của khóa và quy ước xử lý dữ liệu đã sử dụng.

Tự sinh khóa RSA 4096 bit và kiểm tra mã hóa, giải mã thông điệp tiếng Việt:



Khi không cung cấp các tham số đầu vào, chương trình tự sinh hai số nguyên tố xác suất p và q bằng bộ sinh ngẫu nhiên secrets kết hợp phép kiểm tra Miller–Rabin với 40 vòng. Chương trình bảo đảm p khác q, modulus n=p×q có đúng 4096 bit và số mũ công khai e=65537 nguyên tố cùng nhau với phi(n). Số mũ riêng d được tính bằng phép nghịch đảo modulo.

Thực nghiệm sử dụng thông điệp “Thực hành RSA - An toàn mạng máy tính”. Sau khi mã hóa bằng khóa công khai và giải mã bằng khóa riêng, chương trình khôi phục đúng thông điệp ban đầu. Phép kiểm tra e×d mod phi(n) cho kết quả bằng 1. Tổng thời gian sinh khóa và thực hiện phép thử là [điền thời gian] giây.

Kích thước 4096 bit được lựa chọn để minh họa khả năng xử lý số nguyên lớn của chương trình. Đây không phải kích thước lớn nhất tuyệt đối, vì Python hỗ trợ số nguyên có độ dài tùy thuộc tài nguyên máy tính. Khóa được sinh ngẫu nhiên nên các lần chạy có thể tạo ra các giá trị khóa và bản mã khác nhau.

Nhiệm vụ 2.4

Website lựa chọn: **www.amazon.com**

Thu thập chứng chỉ của www.amazon.com và xác định CA phát hành:

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Python\pythoncore-3.14-64\python.exe' .\task24.py fetch --host www.amazon.com
Website: www.amazon.com
Số chứng chỉ máy chủ gửi: 3
c0 subject: CN=www.amazon.com
c0 issuer : CN=Amazon RSA 2048 M04,O=Amazon,C=US
c1 subject: CN=Amazon RSA 2048 M04,O=Amazon,C=US
Thuật toán chữ ký: sha256WithRSAEncryption
Đã lưu c0.pem, c1.pem, chain.txt và download.json vào E:\AnToanMang\lab03\Lab03_RSA_X509\certs
PS E:\AnToanMang\lab03\Lab03_RSA_X509>
```

Chứng chỉ X.509 chứa thông tin về chủ thể, khóa công khai và chữ ký số của nhà phát hành. Phần dữ liệu được ký gọi là TBSCertificate. Để kiểm tra chữ ký trên chứng chỉ máy chủ, cần sử dụng khóa công khai của CA đã phát hành chứng chỉ đó.

Trong nhiệm vụ này, quy trình thực hiện gồm thu thập chứng chỉ từ một máy chủ web, xác định chứng chỉ CA phù hợp, trích xuất khóa công khai của CA, chữ ký và phần TBSCertificate. Sau đó, chương trình Python thực hiện phép tính RSA và đối chiếu cấu trúc chữ ký với giá trị băm của dữ liệu được ký.

Trích xuất modulus n và số mũ e từ khóa công khai của CA:

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Python\pythoncore-3.14-64\python.exe' .\task24.py fetch --host www.amazon.com
Website: www.amazon.com
Số chứng chỉ máy chủ gửi: 3
c0 subject: CN=www.amazon.com
c0 issuer : CN=Amazon RSA 2048 M04,O=Amazon,C=US
c1 subject: CN=Amazon RSA 2048 M04,O=Amazon,C=US
Thuật toán chữ ký: sha256WithRSAEncryption
Đã lưu c0.pem, c1.pem, chain.txt và download.json vào E:\AnToanMang\lab03\Lab03_RSA_X509\certs
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Program Files\Git\mingw64\bin\openssl.exe' x509 -in .\certs\c1.pem -noout -modulus
Modulus=CDE9551E809503A9E317BBEFA15D78F5F27A3800DFC998C75A2EF7F8EBA55E927E7FBFA2A04A830858E9E3EC4071D06B258125D067BF85B3DFADEA448E17521F6CC01149D754E76772EA38CBC7996AB6FA5A00D103ED39F5DD2767C704BAC1AE782D27F181D29E68940558AD2660BD018FD7A45FF1D76D372C44121626DC7BA198D8BEBD2780AB5B4B6093C0BD1097A274E959D483C58EA1BDD9B196A4D88BCD80B415D27E9190E7C1C7334870D02E9A41A92086BC168B97C6B252CEAB3A6298D308B13EDCA9AD06271E897CAA89AA641036994EDA1F27F59B05260068C287F4D69A5DA0B01683A71A30E334A26FD520B29A6B7D05F56FEBF108DA87B96D1959
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Program Files\Git\mingw64\bin\openssl.exe' x509 -in .\certs\c1.pem -text -noout | Select-String -Pattern 'Public Key Algorithm','Public-Key','Exponent'

            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Exponent: 65537 (0x10001)
```

Kết quả thu thập cho thấy máy chủ gửi ba chứng chỉ và chứng chỉ máy chủ có chủ thể CN=www.amazon.com. Nhà phát hành của chứng chỉ máy chủ là CN=Amazon RSA 2048 M04,O=Amazon,C=US, trùng với trường chủ thể của chứng chỉ CA được chọn.

Chứng chỉ máy chủ được lưu thành c0.pem và chứng chỉ CA được lưu thành c1.pem. Thuật toán chữ ký trên chứng chỉ máy chủ là sha256WithRSAEncryption, phù hợp với việc xác minh bằng RSA kết hợp SHA-256 trong nhiệm vụ này. Khóa công khai dùng để kiểm tra chữ ký sẽ được trích từ c1.pem.

Thuật toán và giá trị chữ ký của CA trên chứng chỉ máy chủ:

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Program Files\Git\mingw64\bin
\openssl.exe' x509 -in .\certs\c0.pem -text -noout | Select-Object -Last 2
4
                    Extensions: none
                    Signature : ecdsa-with-SHA256
                                30:45:02:20:31:D5:2E:4C:CA:35:6B:37:A4:07:
35:91:
                                37:9C:42:A7:88:C1:5A:0F:BF:5E:CA:AD:6C:AA:
EE:7F:
                                48:A9:54:C9:02:21:00:9D:1C:F9:23:56:5D:E4:
FD:E7:
                                EB:6D:F3:54:0E:B4:46:3E:B9:A7:C6:34:50:9F:
C0:99:
                                5C:DB:6F:97:17:81:C4
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        57:7f:a3:94:9b:7e:19:4d:5e:8d:90:f3:ab:48:18:bd:8e:6d:
        1f:49:82:bb:2b:af:09:fc:ef:99:df:42:94:f6:b8:76:13:44:
        a0:b5:fd:36:91:73:7c:66:30:b3:a4:88:02:ae:aa:8d:d4:83:
        e5:49:37:0c:67:0d:fd:c0:ad:1e:2f:75:a4:c8:82:30:83:cf:
        8e:c9:d7:d7:0b:5b:0a:70:a8:f7:7f:c6:60:87:48:ac:fa:b6:
        67:48:ba:a1:24:1d:ee:94:25:3f:98:d4:4f:48:5c:44:42:76:
        9d:53:13:2d:cc:da:38:40:09:d9:f2:10:6d:6c:be:b0:8c:c8:
        10:4b:b1:e6:a6:3a:ce:75:de:9d:9d:27:c2:f9:13:00:b9:70:
        03:af:0e:21:c6:25:16:e1:b9:9c:86:63:f2:8d:79:9d:00:2b:
        e3:50:a7:bc:b4:72:61:5d:49:ab:59:fe:7f:5b:66:8d:e1:49:
        41:3c:e0:6f:2c:ce:2a:a4:e8:36:f0:89:aa:64:df:dc:d6:db:
        9e:26:49:bf:a0:6a:06:76:b4:df:b0:d6:b2:51:ed:be:9e:78:
        8d:34:e1:58:43:43:81:cc:e0:0b:4d:cc:de:b7:19:bf:4d:7b:
        58:7f:69:10:2f:62:bf:4e:7a:83:0d:94:4c:ef:c3:3f:de:d5:
        35:c5:da:e2
```

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Python\pythoncore-3.14-64\python.exe' .\task24.py inspect
c0 subject: CN=www.amazon.com
c0 issuer : CN=Amazon RSA 2048 M04,O=Amazon,C=US
c1 subject: CN=Amazon RSA 2048 M04,O=Amazon,C=US

KHÓA CÔNG KHAI CỦA CA
n (HEX) = CDE9551E809503A9E317BBEFA15D78F5F27A3800DFC998C75A2EF7F8EBA55E927E7FBFA2A04A830858E9E3EC4071D06B258125D067BF85B3DFADEA448E17521F6CC01149D754E76772EA38CBC7996AB6FA5A00D103ED39F5DD2767C704BAC1AE782D27F181D29E68940558AD2660BD018FD7A45FF1D76D372C44121626DC7BA198D8BEBD2780AB5B4B6093C0BD1097A274E959D483C58EA1BDD9B196A4D88BCD80B415D27E9190E7C1C7334870D02E9A41A92086BC168B97C6B252CEAB3A6298D308B13EDCA9AD06271E897CAA89AA641036994EDA1F27F59B05260068C287F4D69A5DA0B01683A71A30E334A26FD520B29A6B7D05F56FEBF108DA87B96D1959
e = 65537 ; số bit n = 2048

CHỮ KÝ CỦA CA TRÊN c0 (HEX)
577fa3949b7e194d5e8d90f3ab4818bd8e6d1f4982bb2baf09fcef99df4294f6b8761344a0b5fd3691737c6630b3a48802aeaa8dd483e549370c670dfdc0ad1e2f75a4c8823083cf8ec9d7d70b5b0a70a8f77fc6608748acfab66748baa1241dee94253f98d44f485c4442769d53132dccda384009d9f2106d6cbeb08cc8104bb1e6a63ace75de9d9d27c2f91300b97003af0e21c62516e1b99c8663f28d799d002be350a7bcb472615d49ab59fe7f5b668de149413ce06f2cce2aa4e836f089aa64dfdcd6db9e2649bfa06a0676b4dfb0d6b251edbe9e788d34e158434381cce00b4dccdeb719bf4d7b587f69102f62bf4e7a830d944cefc33fded535c5dae2
Độ dài chữ ký = 256 byte

PHẦN TBSCertificate
Offset = 4 ; hl = 4 ; l = 1707
Độ dài TBS = hl + l = 1711 byte
TBS từ OpenSSL khớp bộ đọc DER = True
SHA-256(TBS) = 2086440c0eaa3387cea8264d8fb5d54238aaa590d0641c81b3761be0399452aa
Các tệp đã lưu trong E:\AnToanMang\lab03\Lab03_RSA_X509\certs
PS E:\AnToanMang\lab03\Lab03_RSA_X509>
```

Cấu trúc ASN.1 của chứng chỉ và giá trị SHA-256 của TBSCertificate:

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Program Files\Git\mingw64\bin
\openssl.exe' asn1parse -i -in .\certs\c0.pem | Select-Object -First 12
    0:d=0  hl=4 l=1987 cons: SEQUENCE
    4:d=1  hl=4 l=1707 cons:  SEQUENCE
    8:d=2  hl=2 l=   3 cons:   cont [ 0 ]
   10:d=3  hl=2 l=   1 prim:    INTEGER           :02
   13:d=2  hl=2 l=  16 prim:   INTEGER           :05660C2EA856E3D523953E93
9E886B5D
   31:d=2  hl=2 l=  13 cons:   SEQUENCE
   33:d=3  hl=2 l=   9 prim:    OBJECT            :sha256WithRSAEncryption
   44:d=3  hl=2 l=   0 prim:    NULL
   46:d=2  hl=2 l=  60 cons:   SEQUENCE
   48:d=3  hl=2 l=  11 cons:    SET
   50:d=4  hl=2 l=   9 cons:     SEQUENCE
   52:d=5  hl=2 l=   3 prim:      OBJECT            :countryName
PS E:\AnToanMang\lab03\Lab03_RSA_X509> Get-FileHash .\certs\c0_body.bin -A
lgorithm SHA256

Algorithm       Hash
---------       ----
SHA256          2086440C0EAA3387CEA8264D8FB5D54238AAA590D0641C81B3761B...


PS E:\AnToanMang\lab03\Lab03_RSA_X509>
```

Phần TBSCertificate được xác định bắt đầu tại offset 4, có phần đầu dài 4 byte và nội dung dài 1707 byte. Tổng độ dài dữ liệu được ký là 1711 byte, bao gồm cả phần đầu mã hóa DER. Chương trình trích dữ liệu này vào tệp c0_body.bin và tính giá trị băm SHA-256.

Kết quả so sánh cho thấy phần TBSCertificate được trích bằng bộ đọc DER của chương trình Python trùng khớp với dữ liệu do OpenSSL trích xuất. Chữ ký của CA trên chứng chỉ máy chủ có độ dài 256 byte và được lưu riêng để phục vụ phép xác minh RSA. Giá trị SHA-256 được tính trên TBSCertificate, không phải trên toàn bộ chứng chỉ hoặc văn bản PEM.

```
PS E:\AnToanMang\lab03\Lab03_RSA_X509> & 'C:\Users\Win\AppData\Local\Python\pythoncore-3.14-64\python.exe' .\task24.py verify
Thuật toán: sha256WithRSAEncryption / PKCS#1 v1.5
c0.issuer == c1.subject: True
Số bit khóa công khai CA: 2048
EM = S^e mod n, độ dài: 256 byte
Đầu EM: 0001ffffffffffffffffffffffffffff
SHA-256 tính từ TBS: 2086440c0eaa3387cea8264d8fb5d54238aaa590d0641c81b3761be0399452aa
32 byte cuối EM   : 2086440c0eaa3387cea8264d8fb5d54238aaa590d0641c81b3761be0399452aa
Khớp TOÀN BỘ padding + DigestInfo + hash: True
CHỮ KÝ HỢP LỆ: True
Sau khi đổi 1 bit của TBS, chữ ký hợp lệ: False
Phạm vi: kiểm tra chữ ký; chưa kiểm tra đầy đủ chuỗi tin cậy, tên miền, thời hạn và thu hồi.
PS E:\AnToanMang\lab03\Lab03_RSA_X509>
```

Chương trình sử dụng khóa công khai RSA 2048 bit của CA Amazon RSA 2048 M04 để xác minh chữ ký trên chứng chỉ của www.amazon.com. Phép tính EM=S^e mod n khôi phục một khối dữ liệu dài 256 byte. Giá trị SHA-256 tính trực tiếp từ TBSCertificate trùng với giá trị băm chứa trong khối chữ ký được khôi phục.

Chương trình đồng thời kiểm tra toàn bộ cấu trúc PKCS#1 v1.5, bao gồm padding, DigestInfo và giá trị băm. Kết quả kiểm tra trả về True, xác nhận chữ ký trên chứng chỉ máy chủ hợp lệ đối với khóa công khai CA đã chọn.

Khi chương trình thay đổi một bit của bản sao TBSCertificate nhưng giữ nguyên chữ ký, kết quả xác minh chuyển thành False. Thực nghiệm minh họa khả năng phát hiện dữ liệu được ký đã bị thay đổi.

Phạm vi kiểm tra trong nhiệm vụ này là xác minh chữ ký. Để đánh giá đầy đủ độ tin cậy của chứng chỉ trong kết nối TLS, cần kiểm tra thêm chuỗi chứng chỉ tới CA gốc tin cậy, tên miền, thời hạn và tình trạng thu hồi.

Kết luận

Qua nhiệm vụ 2.1, chương trình đã thực hiện tạo cặp khóa RSA, mã hóa và giải mã dữ liệu dạng số và chuỗi, giải các bản mã cho trước và thử nghiệm sinh khóa ngẫu nhiên. Các kết quả cho thấy việc xử lý đúng mã hóa ký tự, thứ tự byte và ranh giới khối là cần thiết để khôi phục chính xác thông điệp.

Qua nhiệm vụ 2.4, chương trình đã thu thập chứng chỉ từ máy chủ web, trích xuất khóa công khai của CA, chữ ký và TBSCertificate, sau đó xác minh chữ ký bằng phép lũy thừa modulo RSA. Chữ ký hợp lệ đối với dữ liệu nguyên vẹn và không hợp lệ khi một bit của dữ liệu bị thay đổi. Kết quả minh họa vai trò của hàm băm và chữ ký số trong kiểm tra tính toàn vẹn của dữ liệu.